6 STRING HANDLING

Strings Start

Create a program called **strings_start** that first inputs a string from the user. Then the program should print the following: the string in all small letters, the string in all capital letters, and the length of the string.

Example output:

```
Enter a string: First String Exercise

first string exercise
FIRST STRING EXERCISE
21 characters
```

Strings of Pearls

Create a program called **strings_of_pearls** that first inputs strings from the user until he/she decides to quit by entering the text "**quit**". Then the program should count how many times the user enters the string "pearl". The program should test the entered sting in case-insensitive way.

If the user does not enter anything else but "quit", the program should not print anything else but "Bye!".

Example output:

```
Enter first string: Orange
Enter next string: Pearl
Enter next string: Pear
Enter next string: PEARL
Enter next string: pearl
Enter next string: quit

3 pearls
Bye!
```

Strings Digit Sum

Create a program called **strings_digit_sum** that first inputs a string from the user. Then the program should print the sum of all digits that are included in the inputted string. If there are no digits in the string, the program should print 0.

Example output:

```
Enter a string: abc1234

The sum of digits is 10
```

Strings Surnames

Create a program called **strings_surnames**. The program should first ask the user how many surnames the user will enter. Then the program should input the surnames from the user. Finally, the program should print distinct surnames as shown in the example output.

Example output:

```
How many surnames? 6
Enter a surname: sheeRAN
Enter a surname: Eilish
Enter a surname: Niinistö
Enter a surname: marin
Enter a surname: Mars
Enter a surname: niinistÖ

Eilish Marin Mars Niinistö Sheeran
```

Strings Subset

Create a program called **strings_subset** that first inputs two strings from the user. The program should check if all the characters in the second string are included in the first string.

The program should print "Subset" if this is true. Otherwise, the program should print "Not subset". If the second string is empty, the program should print "Nothing to be checked".

Hint: A boolean variable (initialized to True) and a loop is useful in this exercise.

Example output:

```
Enter first string: document
Enter second string: code
Subset
```

Strings Box

Create a program called **strings_box** that first inputs a string. Then the program should print the inputted string in a in a "box" as shown in the example output.

NB! You should not write a loop in your program.

Example output:

```
Enter a string: Hello
---------
| Hello |
---------
```

Strings Slicing 1

Create a program called **strings_slicing_1** that first inputs a string. Then the program should print the following: ***first two characters*** *in the string* and ***every other character*** *in the string*.

NB! You should use **slicing** in this exercise.

Example output:

```
Enter a string: heavy
he
hay
```

Strings Slicing 2

Create a program called **strings_slicing_2** that first inputs a string. Then the program should print the ***next to last character*** in the string. If the string has less than two characters, the program should print "Too short string".

NB! You should use **slicing** in this exercise.

Example output:

```
Enter a string: exercise
s
```

Strings Slicing 3

Create a program called **strings_slicing_3** that first inputs a string. Then the program should print all substrings of the string, which start from the first position in the string. The substrings should be printed as shown in the example output.

NB! You should use **slicing** in this exercise.

Hint: A loop is useful here.

Example output:

```
Enter a string: Python
P
Py
Pyt
Pyth
Pytho
Python
```

Strings Slicing 4

Create a program called **strings_slicing_4** that first inputs a string. Then the program should detect if the string reads the same backward as forward. If this is true, the program should print "Yes". Otherwise, the program should print "No".

For example, "12XOX21" reads the same backward as forward.

NB! You should use **slicing** in this exercise. After inputting a string, you can check it in a single if statement.

Example output:

```
Enter a string: stats
Yes
```

Strings Slicing 5

Create a program called **strings_slicing_5** that first inputs a string and a character from the user. The program should examine the inputted string and print all 4-character substrings, which start with the inputted character. Substrings can overlap. If no such substring exists in the string, the program should not print anything more.

NB! You should use a loop and slicing in this exercise.

Example output:

```
Enter a string: sunshine
Enter a character: s
suns
shin
```

Strings Same Characters

Create a program called **strings_same_characters** that inputs two strings. The program should first remove blanks from the strings and then detect if the strings contain exactly the same set of characters. The program should print "Same characters" if this is true. Otherwise, the program should print "Different characters".

The characters are not required to be in the same order in both strings. For example, "A1234A" and "AA2413" contain the same set of characters. "BORROW" and "BROW" do not contain the same set of characters.

NB! Slicing is ***not*** needed in this exercise.

Hint: You should ***not*** write a loop in this exercise. After inputting two strings, you can check them in a single if statement.

Example output:

```
Enter first string: A1234A
Enter second string: AA2413
Same characters
```

Strings Duplicate Characters

Create a program called **strings_duplicate_characters** that first inputs a string from the user. The program should check if any character occurs more than once in the string. If there are no duplicate characters in the string, the program should print "No duplicates". Otherwise, the program should print "Contains duplicate(s)".

NB! You should ***not*** write a loop is here. We can solve this problem using a set.

Example output:

```
Enter a string: benchmark
No duplicates
```

Strings Circular Primes (*This is an **optional** more advanced grade level 5 exercise*)

Create a program called **strings_circular_prime** that has the following three functions: **main**, **is_prime**, and **is_circular_prime**.

The **is_prime** function should take an integer as argument and return **True** if the integer is a *prime number*. Otherwise, the function should return **False**. The function should not input anything from the user or print anything.

The **is_circular_prime** function should take an integer as argument and return **True** if the integer is a *circular prime number*. Otherwise, the function should return **False**. The function should not input anything from the user or print anything.

A *prime number* (prime) is an integer greater than 1 whose only factors are 1 and itself.

A *circular prime number* is a prime number that remains prime on any cyclic rotation of its digits in base 10. For example,

113 is a *circular prime* because 113, 131, and 311 are primes.
41 is a prime but is not a *circular prime* because its circular rotation 14 is not a prime.

The *main function* should first ask the user to enter a positive integer. Then the main function should print the integer and either the text "is a circular prime" or "is not a circular prime". The main function should call the **is_circular_prime** function to determine if a number is a circular prime or not.

Example output:

```
Enter a positive integer: 19937
19937 is a circular prime
```